



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Tarea 3

ALUMNO

Pérez Takayanagui Louis Emilio
322311419

MATERIA

Estructura y Programación de Computadoras

19 de octubre de 2026

Operaciones ADC y SBB

1. Introducción

En esta tarea usaremos las operaciones `ADC` (suma con acarreo) y `SBB` (resta con *borrow*) para realizar una suma y una resta con dos números de 8 *words* cada uno y comprobar que sí funcionan correctamente las operaciones.

2. Planteamiento

Se implementará un ciclo de 8 iteraciones para cada operación, evitando la redundancia de código secuencial mediante el uso de la instrucción `loop`.

Los operandos definidos para la práctica corresponden a cantidades de 128 bits expresadas en hexadecimal:

- `num1` = `6790 BF93 C642 078A 1D53 40B7 E50A 75B9`$_h$
- `num2` = `F58D 75B3 07E3 5F31 70D7 85E2 076C BC40`$_h$

Para la suma, en cada iteración se transfieren 16 bits (1 *word*) del arreglo `num1` al registro `AX` y los correspondientes 16 bits de `num2` al registro `DX`, utilizando el registro `BX` como índice base. A continuación, se ejecuta la instrucción `ADC` para realizar la suma considerando el acarreo previo y el resultado se almacena en memoria. El índice `BX` se incrementa mediante dos instrucciones `inc bx` consecutivas, permitiendo avanzar al siguiente elemento tipo *word* sin alterar el valor de la bandera de acarreo (*Carry Flag*).

Una vez finalizado el ciclo, se ejecuta una última instrucción `ADC` sobre la novena palabra del arreglo en memoria con el fin de capturar el acarreo final de la operación de 128 bits.

Para la resta se sigue una estructura análoga, sustituyendo la suma con acarreo por la instrucción `SBB` (*Subtract with Borrow*) para restar palabra por palabra con préstamo, y aplicando un último `SBB` al concluir el ciclo para registrar el préstamo final en memoria.

Cada número se almacena en memoria en el orden en que se define con `dw`: la primera palabra (`6790h` en `num1` y `0F58Dh` en `num2`) es la menos significativa, por lo que se procesa primero y el acarreo (o préstamo) se propaga hacia las palabras siguientes. Por esta razón, en los Cuadros 1 y 2 los números se escriben con `75B9h` como palabra más significativa.

Cabe señalar que, en la resta, la instrucción `SBB` final opera sobre la novena palabra del arreglo `resta`, inicializada en `0000h`. Si la última resta de words generó préstamo (CF = 1), se calcula $0000h - 0 - 1 =$ `FFFFh`, por lo que dicha palabra queda en `FFFFh` y no en `0001h`. Este valor es la extensión en complemento a 2 del resultado negativo e indica que `num1` < `num2`. Si no hubiera préstamo, la palabra permanecería en `0000h`. En cambio, en la suma el `ADC` final deja `0001h` cuando existe acarreo.

3. Diagrama de flujo

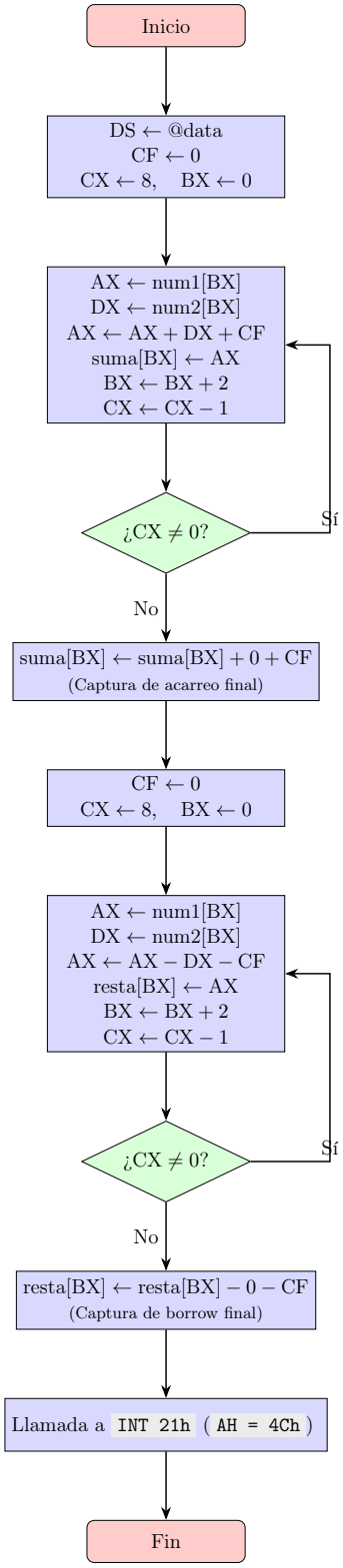


Figura 1: Diagrama de flujo de las operaciones de precisión extendida.

4. Justificación

La arquitectura x86 en modo real limita las operaciones nativas al tamaño de sus registros, por lo que manejar números de 128 bits requiere descomponer el cálculo en bloques de 16 bits. Las instrucciones `ADC` y `SBB` son fundamentales para este propósito, ya que integran directamente la bandera de acarreo (*Carry Flag*) de la iteración previa. Asimismo, se optó por una estructura iterativa con `loop` y el avance mediante `inc bx` para compactar el código y evitar la sobreescritura accidental del acarreo que causaría una instrucción como `add bx, 2`.

Inspeccionamos el programa con el *Turbo Debugger*, en el que se verifican los valores obtenidos, los cuales coinciden con el cálculo teórico mostrado a continuación.

Carry:	**0**	**0**	**0**	**0**	**0**	**1**	**1**	**0**
	75B9	E50A	40B7	1D53	078A	C642	BF93	6790 h
+	BC40	076C	85E2	70D7	5F31	07E3	75B3	F58D h
	1 31F9	**EC76**	**C699**	**8E2A**	**66BB**	**CE26**	**3547**	**5D1D h**

Cuadro 1: Suma con acarreo

Borrow:	**0**	**1**	**1**	**1**	**0**	**0**	**1**	**0**
	75B9	E50A	40B7	1D53	078A	C642	BF93	6790 h
−	BC40	076C	85E2	70D7	5F31	07E3	75B3	F58D h
	B979	**DD9D**	**BAD4**	**AC7B**	**A859**	**BE5F**	**49DF**	**7203 h**

Cuadro 2: Resta con *borrow* (borrow final = 1, que en memoria queda como `FFFFh`)

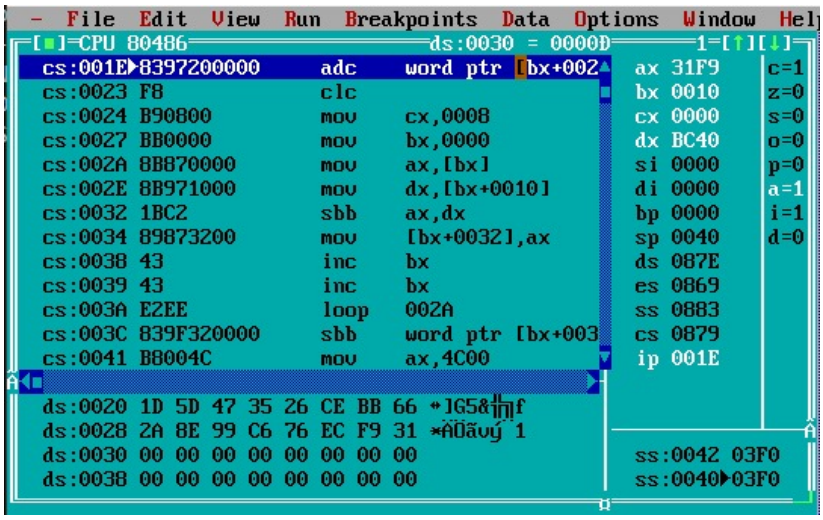


Figura 2: Captura del Turbo Debugger del resultado de la suma hexadecimal.

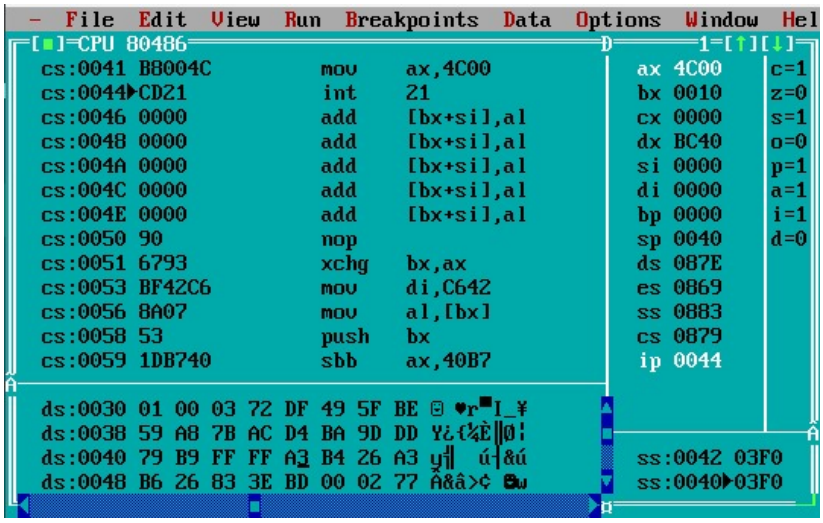


Figura 3: Captura del Turbo Debugger del resultado de la resta hexadecimal.

**Nota: Los valores en el Turbo Debugger aparecen en notación Little Endian.*

5. Código del programa

```
title "Sumas y restas grandes"
.model small
.386
.stack 64

.data
    num1 dw 6790h, 0BF93h, 0C642h, 078Ah, 1D53h, 40B7h, 0E50Ah, 75B9h
    num2 dw 0F58Dh, 75B3h, 07E3h, 5F31h, 70D7h, 85E2h, 076Ch, 0BC40h
    suma dw 0h, 0h, 0h, 0h, 0h, 0h, 0h, 0h, 0h
    resta dw 0h, 0h, 0h, 0h, 0h, 0h, 0h, 0h, 0h

    .code
Inicio:
    mov ax, @data
    mov ds, ax; segmento de datos

    clc
    mov cx, 8; preparamos registro cx para loop
    mov bx, 0; preparamos bx para usarlo como indice

ItSuma:
    mov ax, num1[bx]
    mov dx, num2[bx]
    adc ax, dx; suma con acarreo
    mov suma[bx], ax; guardamos el nuevo valor
    inc bx
    inc bx; incrementamos bx dos veces porque son datos tipo word
    loop ItSuma

    adc suma[bx], 0; acarreo final

    clc
    mov cx, 8; volvemos a preparar cx
    mov bx, 0; reiniciamos el indice

ItResta:
    mov ax, num1[bx]
    mov dx, num2[bx]
    sbb ax, dx; resta con borrow
    mov resta[bx], ax; guardamos el nuevo valor
    inc bx
    inc bx
    loop ItResta

    sbb resta[bx], 0; borrow final


Exit:

    mov ax, 4C00h
    int 21h
    end Inicio
```

Listing 1: Código del programa